

Object-Oriented Programming

1. What is OOP?

- Object-Oriented Programming (OOP) is a programming approach where a program is designed using objects that contain data and functions together.
- In OOP, we try to represent real-world things as **objects**.

Student

↓

Data → name, age, marks

Function → study(), attend_exam()

Ex

```
class Student:

    def study(self):
        print("Student is studying")
```

2. What is POP?

- POP means Procedure-Oriented Programming
- It focuses mainly on functions/procedures and the sequence of operations.

```
def add_student():
    pass

def calculate_marks():
    pass

def display_student():
    pass
```

3. OOP vs POP

POP	OOP
Procedure-oriented programming	Object-oriented programming
Focuses on functions/procedures	Focuses on objects
Data and functions are generally separate	Data and methods are combined in objects
Less suitable for very large complex systems	Suitable for large and complex systems
Code reuse is comparatively limited	Supports strong code reuse
Example: C	Examples: Python, Java, C++

4. Six Principles of OOP

- 1) Class
- 2) Object
- 3) Encapsulation
- 4) Abstraction
- 5) Inheritance
- 6) Polymorphism

5. What is a Class?

- A class is a blueprint or template used to create objects.

```
class Student:  
    pass
```

6. What is an Object?

- An object is an instance of a class

```
student1 = Student()  
student2 = Student()
```

7. Class vs Object

Class	Object
Blueprint/template	Instance of a class
Defines structure/behavior	Represents an actual entity
Does not represent one specific entity	Represents a specific entity
Used to create objects	Created from a class

8. What is an Attribute?

- An attribute is a variable that belongs to an object or class and stores its data/state.

```
class Student:

    def __init__(self, name, age):
        self.name = name
        self.age = age
```

9. What is a Method?

- A method is a function defined inside a class that describes the behavior of an object.

```
class Student:

    def study(self):
        print("Student is studying")
```

10. What is __init__()

- __init__() is a special method that is automatically called when an object is created. It is commonly used to initialize the object's attributes.

```
class Student:

    def __init__(self, name, age):
        self.name = name
        self.age = age
```

11. self Keyword in Python

- self is a reference to the current object of a class. It is used inside instance methods to access the attributes and methods of that particular object.
- Suppose we create two objects:

```
class Student:

    def __init__(self, name, age):
        self.name = name
        self.age = age

student1 = Student("Rahul", 20)
student2 = Student("Amit", 22)
```

12. Naming Conventions in Python

- Naming conventions are recommended rules for naming variables, functions, classes, methods, constants, and other identifiers so that Python code is readable and consistent.

- 1) Class Names — PascalCase
- 2) Variable Names — snake_case
- 3) Method Names — snake_case
- 4) Constants — UPPER_CASE
- 5) Private-Style Names — __name